Cybersecurity Intrusion Detection — Final Report

BELLEPERCHE Grégoire
QUERREC Thomas
Montenegro Loureiro Marco-Antonio
Relut-Vainqueur Xavier

29/04/2026

Contents

1 Introduction **2**
1.1 Problem statement . . . 2
1.2 Approach at a glance . . . 2
1.3 Stack . . . 2

2 Data Preprocessing and Exploratory Data Analysis **3**
2.1 Dataset overview . . . 3
2.2 Class balance (Figure 1) . . . 3
2.3 Numerical features per class (Figures 2 and 3) . . . 3
2.4 Categorical features and the `browser = Unknown` finding (Figure 4) . . . 5
2.5 Correlation structure (Figures 5 and 6) . . . 5
2.6 Feature engineering (Figures 7 and 8) . . . 5
2.7 Final preprocessing pipeline . . . 8

3 Model Training and Comparison **8**
3.1 Classical baselines (Figures 9 and 10) . . . 8
3.2 Deep Learning (Figures 11 and 12) . . . 10
3.3 Ensembles . . . 12
3.4 Champion selection . . . 12

4 Evaluation Metrics **12**
4.1 Metrics chosen . . . 12
4.2 Full comparison table (Figure 20) . . . 12
4.3 Threshold analysis (Figure 21) . . . 13

5 Interpretability and Explainability **14**
5.1 Method choice . . . 14
5.2 Global SHAP (Figures 13, 14, 19) . . . 14
5.3 Individual predictions (Figures 15, 16, 17) . . . 14
5.4 Feature dependence (Figure 18) . . . 17

6 Deployment and Performance Tracking **17**
6.1 Architecture . . . 17
6.2 Champion bundle and startup . . . 22
6.3 The Gradio UI: live SHAP and 4-model comparison . . . 22
6.4 `docker compose up -d` and the reproducibility contract . . . 22
6.5 MLflow tracking . . . 23

7 Reflections and Observations **23**
7.1 Challenges faced . . . 23
7.2 Key takeaways . . . 23
7.3 Limits and future work . . . 24

8 Conclusion **24**

- **Course:** MOD10 Machine Learning, Winter 2026, Concordia
- **Instructor:** Mohammed A. Shehab
- **Group:** 07
- **Project:** Cybersecurity Intrusion Detection (ML + DL + Docker deployment)

1 Introduction

1.1 Problem statement

We built a binary classifier that flags a network session as **Attack** or **Normal** from nine raw features describing the traffic and the user behaviour. As a team of four, we were asked to design the full pipeline end-to-end: exploratory analysis, feature engineering, classical and deep-learning models, interpretability, and a deployable service. We picked **Random Forest as the deployed champion** (F1 = 0.855 on a held-out test set of 1908 sessions). It is served through a FastAPI + Gradio stack and packaged in a single Docker Compose setup.

1.2 Approach at a glance

Rather than chasing the highest single metric, we organised the project as a chain of explicit decisions. Each one was motivated by what we observed at the previous step:

1. **EDA first.** We let the data tell us which features were worth engineering and which we could probably drop. Two findings drove every decision later on: `failed_logins` is the strongest univariate predictor (Pearson 0.36), and `browser_type = Unknown` carries a 73 % attack rate versus a 43 % baseline.
2. **Baselines before deep learning.** We started with a `DummyClassifier` as a sanity floor (F1 = 0). Then Logistic Regression, Random Forest, and XGBoost. Only after that did we train four variants of a Dense Neural Network.
3. **Honest comparison.** When our DNN did not beat the tree-based models, we reported it as such. On a 7 629-sample tabular problem, gradient-boosted and bagged trees are expected to dominate Deep Learning (Grinsztajn et al. 2022), and our numbers reproduced that.
4. **Interpretability in production.** SHAP is not just a notebook artefact for us. It runs *live* in the deployed UI on every prediction.
5. **Deployment that actually launches.** `docker compose up -d` is the one command required, and it starts a single service that serves both the REST API and the Gradio UI on port 8000.

1.3 Stack

The tools we used:

- **Python 3.10+**, `pandas`, `numpy`, `scikit-learn` (pinned to 1.7.2 for cross-version reproducibility)
- **TensorFlow / Keras** for the DNN
- **XGBoost** for gradient boosting
- **SHAP** (TreeExplainer) for global and local explanations
- **FastAPI + Uvicorn** for the REST service
- **Gradio** mounted on the FastAPI app at `/ui` for the interactive UI
- **Docker + Docker Compose** for packaging
- **MLflow** with a file-backed store for experiment tracking (10 runs across notebooks 02, 03 and 05)

---

2 Data Preprocessing and Exploratory Data Analysis

2.1 Dataset overview

The Kaggle *Cybersecurity Intrusion Detection* dataset gave us **9 537 network sessions** with 11 columns. After dropping `session_id` and the `attack_detected` target, we worked with 9 predictor features: five numerical (`network_packet_size`, `login_attempts`, `session_duration`, `ip_reputation_score`, `failed_logins`), three categorical (`protocol_type`, `encryption_used`, `browser_type`) and one binary (`unusual_time_access`).

Key data-quality observations:

- **No duplicates.**
- **One column with missing values:** `encryption_used` has 1 966 NaN (20.6 %). We chose to keep the NaN as an explicit `Unknown` category rather than imputing, so the model could learn from the pattern of missingness itself.
- **No outliers worth dropping.** The numerical features have well-behaved ranges on inspection.

2.2 Class balance (Figure 1)

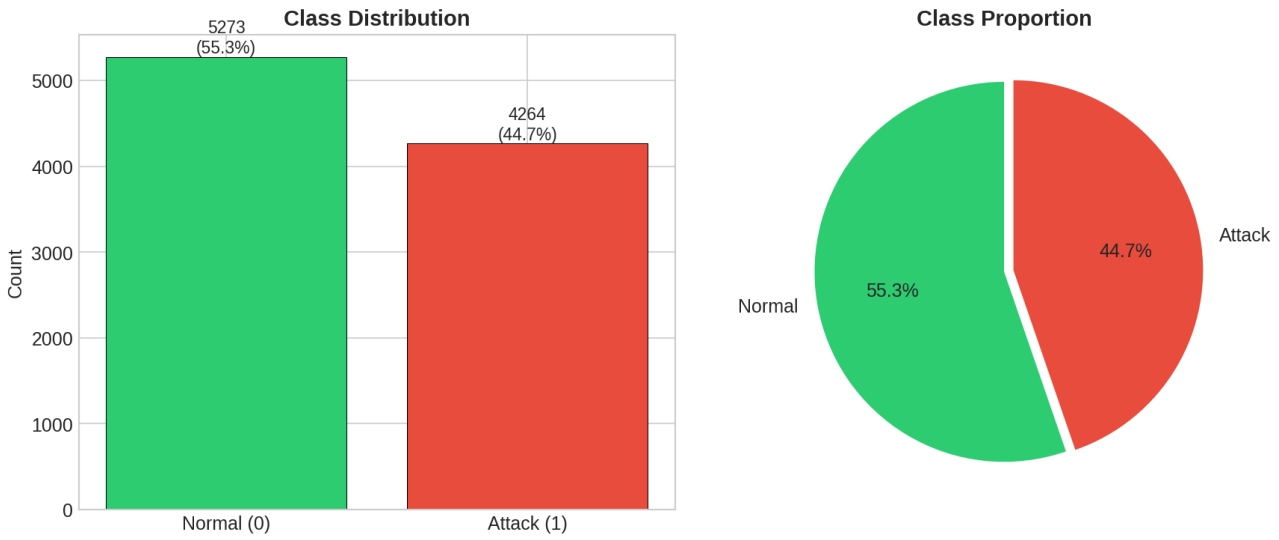

Figure 1: Target distribution. The dataset is moderately imbalanced (Normal 55.3 % vs Attack 44.7 %), so we did not use SMOTE.

The two classes sit at **Normal 5 273 (55.3 %)** and **Attack 4 264 (44.7 %)**, a ratio of 1.24:1. Real intrusion data in production would be closer to 99:1, but this "balanced for teaching" distribution had one concrete consequence for us: we did not apply SMOTE. The class imbalance was not pathological, and synthetic samples would only have added noise. Instead we set `class_weight="balanced"` on Logistic Regression and Random Forest, and `scale_pos_weight = 1.24` on XGBoost. The DNN was trained without re-weighting and its probability outputs were calibrated enough that no correction was needed.

2.3 Numerical features per class (Figures 2 and 3)

These two figures tell a clear story. `failed_logins` and `ip_reputation_score` are the only two numerical features where the distributions of the two classes visibly diverge. `network_packet_size` and `session_duration` are essentially overlapping between classes, which is what motivated the engineered `packet_rate` feature in Section 2.6. We decided early on that any model doing significantly better than a baseline that only used these two features would have to be extracting non-linear interactions.

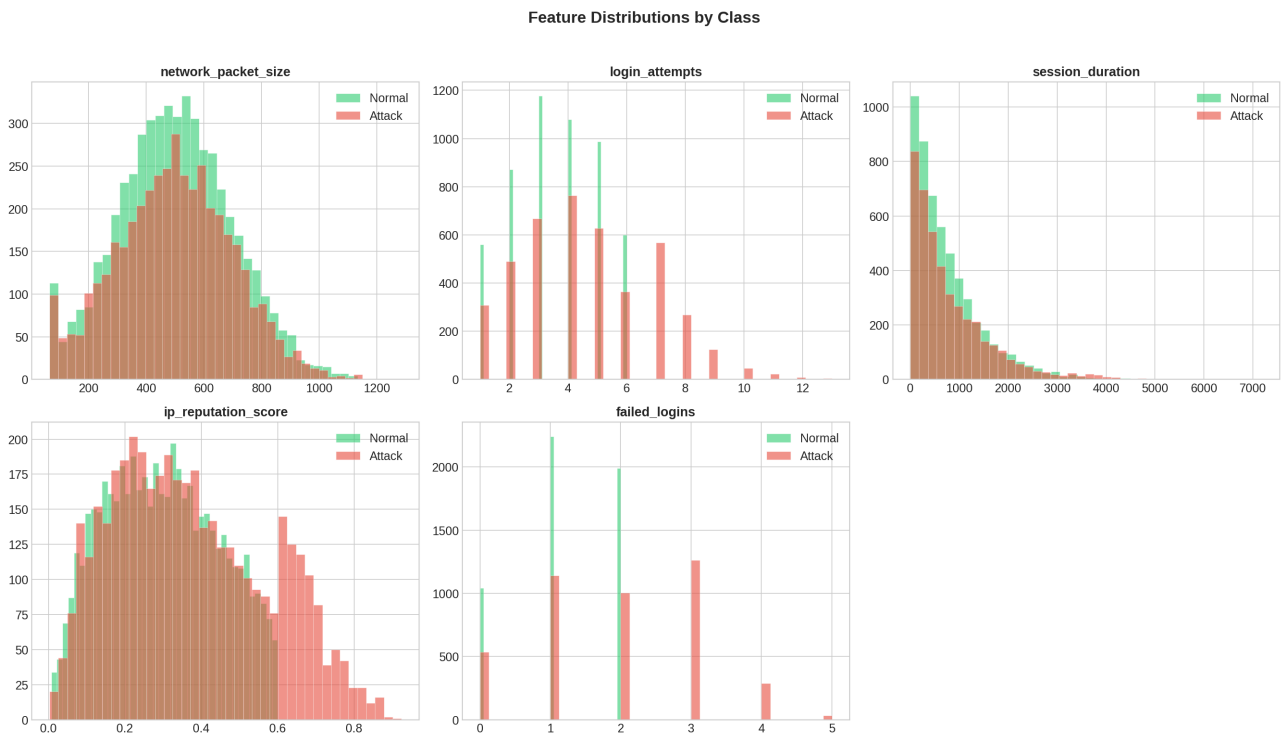

Figure 2: Overlaid histograms of the five numerical features, split by class. Only failed_logins and ip_reputation_score visibly separate the two classes.

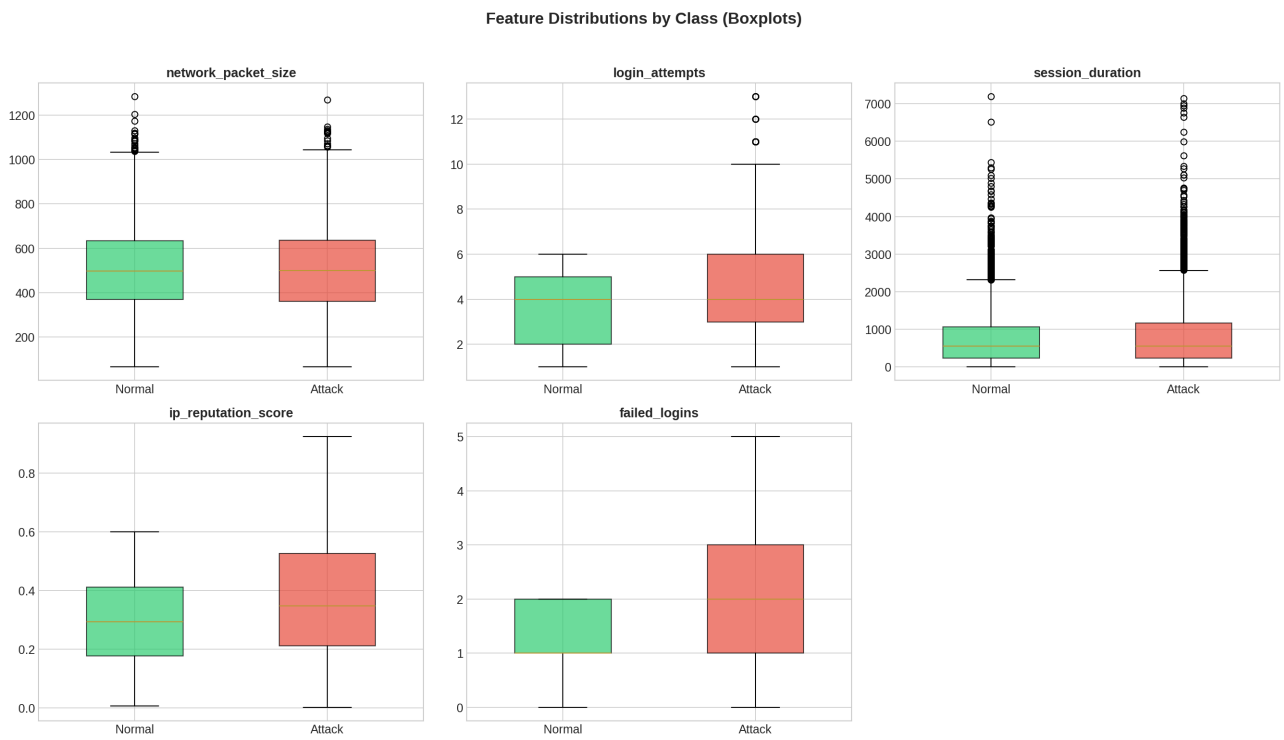

Figure 3: Boxplots of the same five features per class. The previous insight is confirmed: failed_logins and ip_reputation_score are the only two numerical features with meaningfully different distributions between classes.

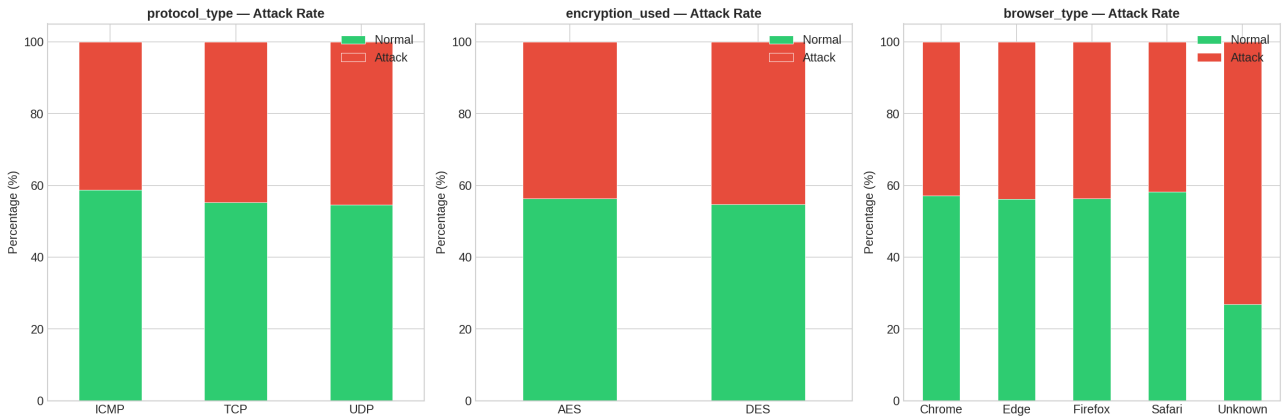

Figure 4: Attack rate per modality for protocol_type, encryption_used and browser_type. The striking finding is browser_type = Unknown reaching 73 % attack rate, versus 42 to 44 % for the four known browsers.

2.4 Categorical features and the `browser = Unknown` finding (Figure 4)

This figure is the single most interesting result of the EDA phase. Protocol and encryption choices carry essentially no signal on their own (41 to 46 % attack rate across modalities). But `browser_type = Unknown`, which in practice covers bots, `curl`, scripts, and anything that does not present a recognised User-Agent, has a **73 % attack rate**. That is close to 1.7× the baseline.

We kept Unknown as a separate one-hot category rather than bucketing it with "Other". Later in the SHAP analysis we confirmed that `browser_Unknown` was consistently in the top features every tree-based model relied on. This was the most actionable finding from the EDA.

2.5 Correlation structure (Figures 5 and 6)

Two things mattered here:

- **No multi-collinearity.** All inter-feature correlations are within ±0.02, so we did not need to drop redundant features or use PCA. We could feed the full set to every model and let each one decide what to use.
- **Non-linear decision boundary.** The pairplot shows the two classes inter-mixed in every 2D projection. This is also why a Logistic Regression, even with `class_weight="balanced"`, plateaus at F1 = 0.73, while tree ensembles climb to 0.85.

2.6 Feature engineering (Figures 7 and 8)

We created four engineered features in `src/utils/feature_engineering.py`:

- `login_fail_ratio = failed_logins / login_attempts` (guarded against division by zero)
- `packet_rate = network_packet_size / session_duration` (guarded the same way)
- `risk_score = 0.4 · ip_reputation_score + 0.3 · unusual_time_access + 0.3 · login_fail_ratio`
- `high_risk_ip = (ip_reputation_score > 0.7)` as an integer flag

The ranking in Figure 8 is what we actually used to decide what to keep. `risk_score`, `high_risk_ip`, and `login_fail_ratio`, all engineered, rank 4th, 5th, and 6th respectively, beating four of the raw features. `packet_rate` ranked dead last, in line with the fact that the features it combines also have essentially zero correlation with the target. We kept it anyway, because tree-based models can still exploit it through interactions that univariate correlation cannot surface.

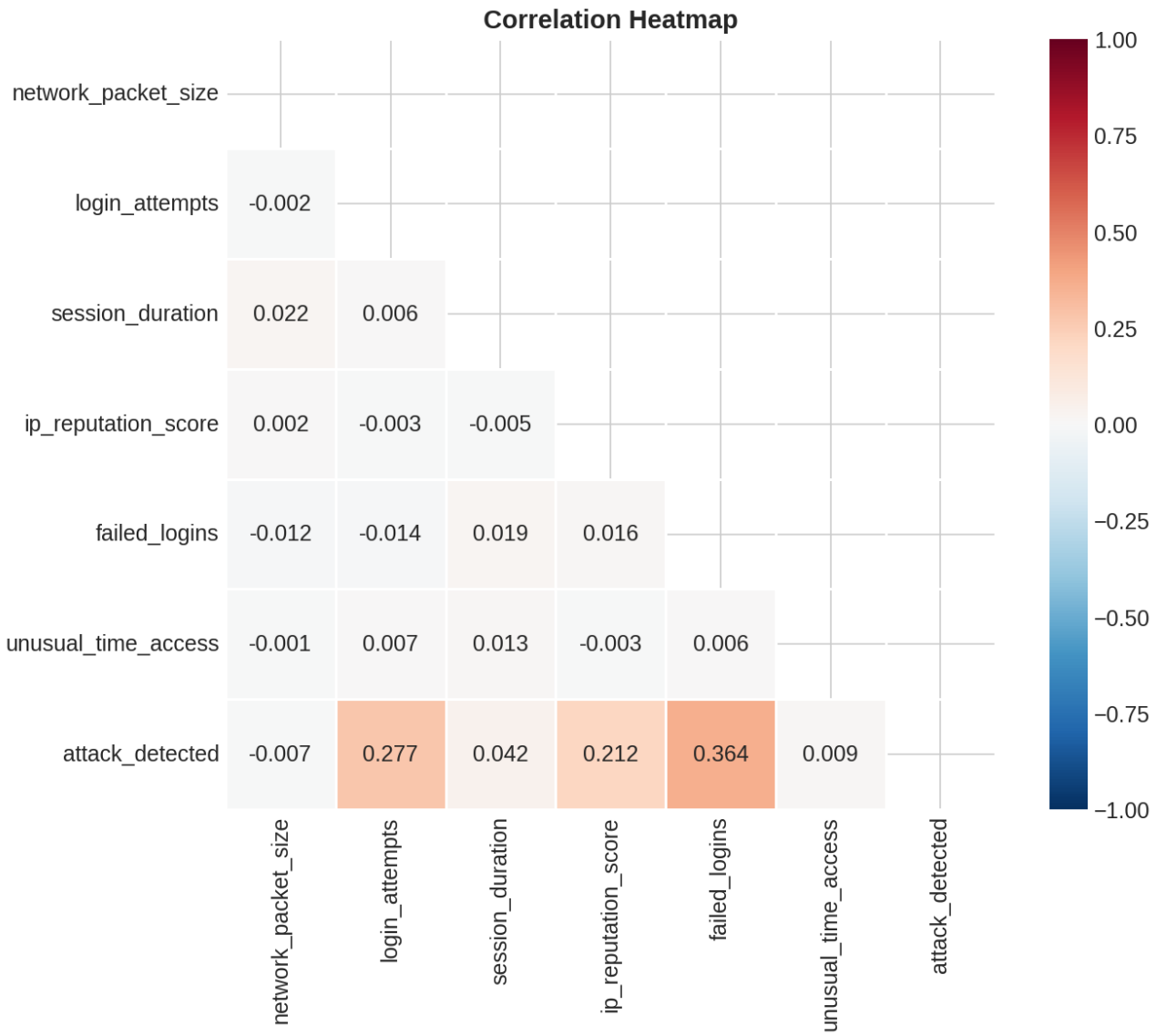

Figure 5: Pearson correlation heatmap. All feature-feature correlations stay within ±0.02 (the features are mutually independent). With the target, failed_logins (+0.36), login_attempts (+0.28) and ip_reputation_score (+0.21) are the only non-trivial signals.

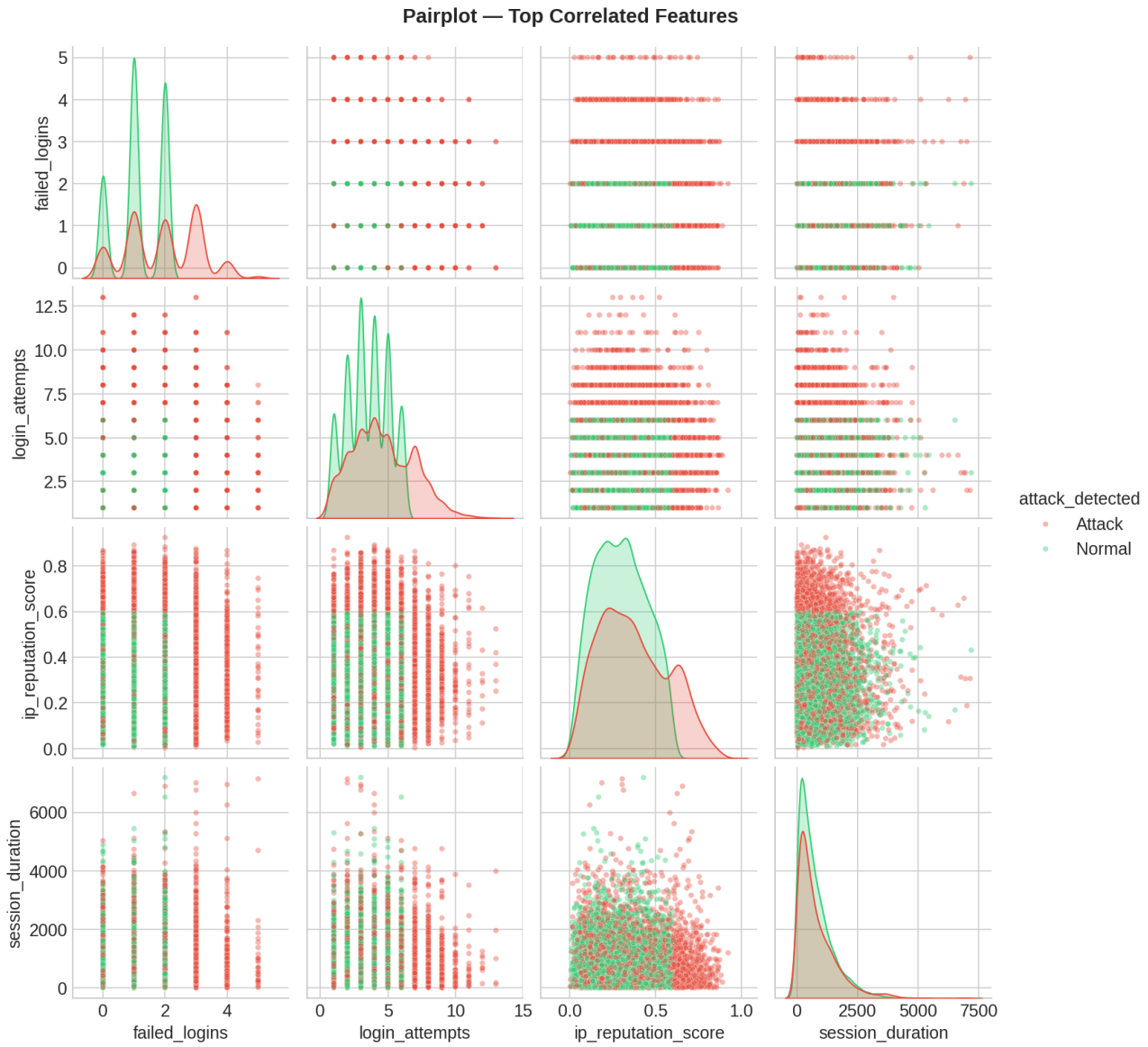

Figure 6: Pairplot of the four strongest predictors (login_attempts, failed_logins, ip_reputation_score, risk_score). The class clusters overlap heavily in every 2D projection. The separation is inherently non-linear in the full feature space.

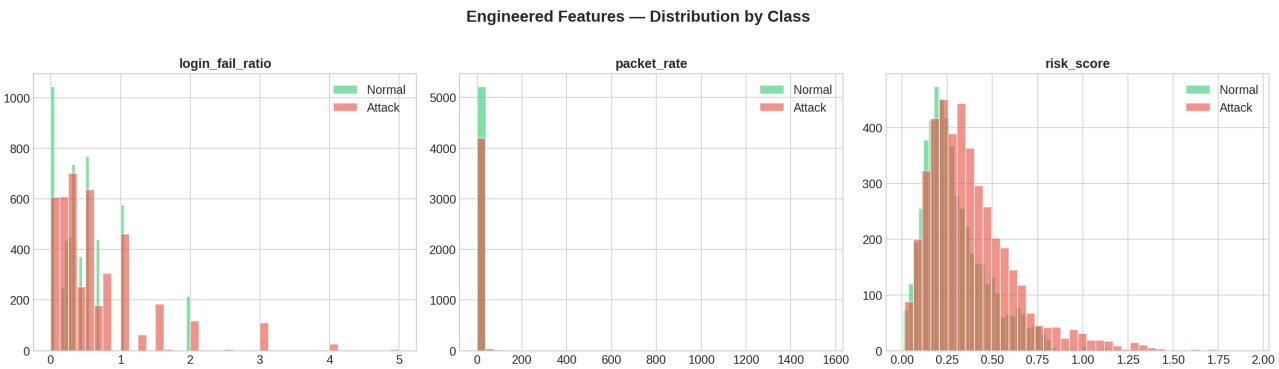

Figure 7: Distributions of the three numerical engineered features per class. login_fail_ratio cleanly separates the classes above ratio ~ 1; risk_score adds a modest but meaningful signal; packet_rate is essentially noise, confirming the EDA.

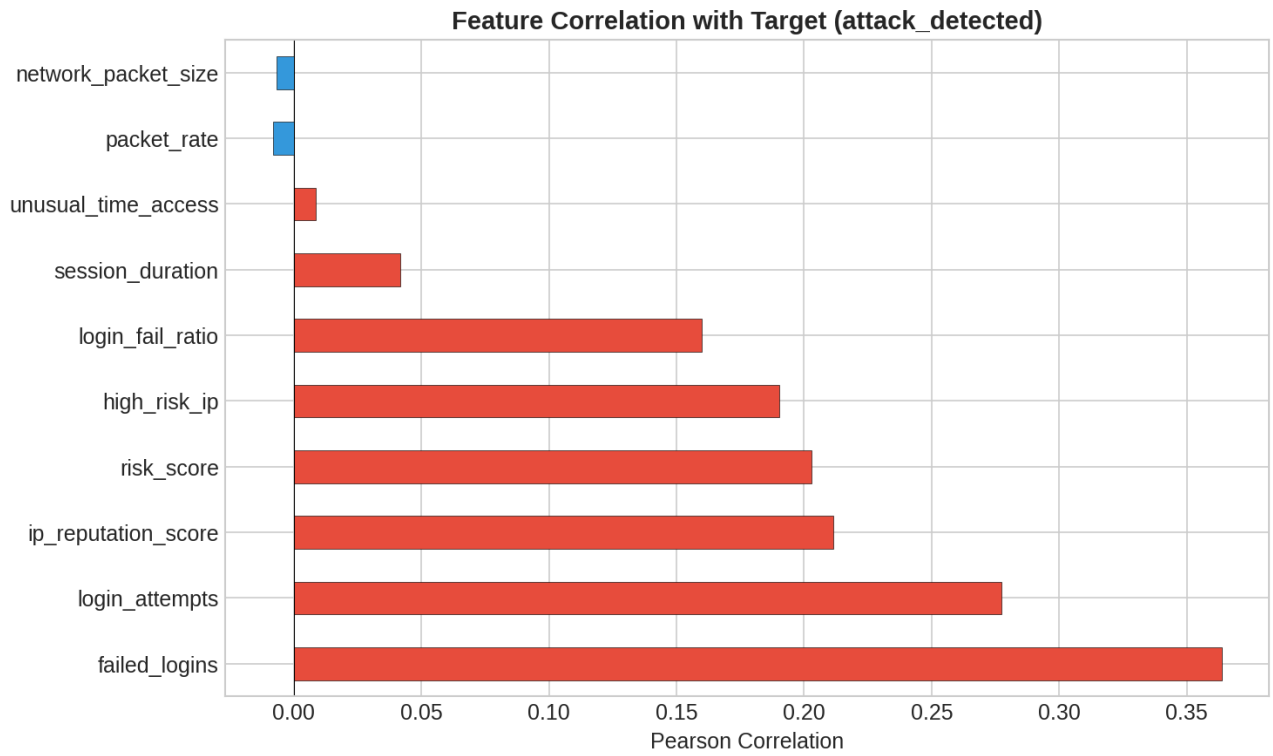

Figure 8: Bar chart of Pearson correlation with the target, ranked. Three out of four engineered features (risk_score, high_risk_ip, login_fail_ratio) land in the top six, validating the engineering decisions.

2.7 Final preprocessing pipeline

The fitted pipeline, wrapped in a scikit-learn `ColumnTransformer`, is:

- **Numerical** (5 raw + 3 engineered = 8 features): `StandardScaler` (Z-Score)
- **Categorical** (`protocol_type`, `encryption_used`, `browser_type`): `OneHotEncoder(drop="first", handle_unknown="ignore")`. The `handle_unknown="ignore"` lets inference-time Unknown values encode as all zeros rather than throw.
- **Binary** (`unusual_time_access` and `high_risk_ip`): pass-through

After the transformer, each session is an 18-column vector. We split 80/20 stratified on the target, which gave us `X_train.shape = (7629, 18)` and `X_test.shape = (1908, 18)`. The random seed (42) is fixed in `numpy`, `scikit-learn` and `tensorflow` for exact reproducibility.

The entire pipeline is serialised to `saved_models/preprocessor.joblib`, so training and inference use the *same* transformer. This avoids training-serving skew by construction.

---

3 Model Training and Comparison

Our strategy was to build up complexity step by step. One baseline per hypothesis, and we only moved on to the next model when the previous one had taught us something. All runs were tracked in MLflow under the experiment `cybersecurity-ids`.

3.1 Classical baselines (Figures 9 and 10)

We trained four baselines:

- **Dummy Classifier** (`strategy="most_frequent"`): the floor. It predicts Normal every time, which gives F1 = 0.0000 and Accuracy = 0.5529. Any model we keep must beat this.
- **Logistic Regression**: tests whether the problem is linearly separable.
- **Random Forest** (`n_estimators=200`, `max_depth=15`, `class_weight="balanced"`): tests whether tree ensembles capture the non-linear boundary we saw in Section 2.5.
- **XGBoost** (`n_estimators=200`, `max_depth=6`, `learning_rate=0.1`, `scale_pos_weight=1.24`): tests whether gradient boosting adds anything over bagging.

All four were logged to MLflow with parameters, metrics (accuracy, precision, recall, F1, ROC-AUC, PR-AUC), train time, inference time, and the fitted model artefact.

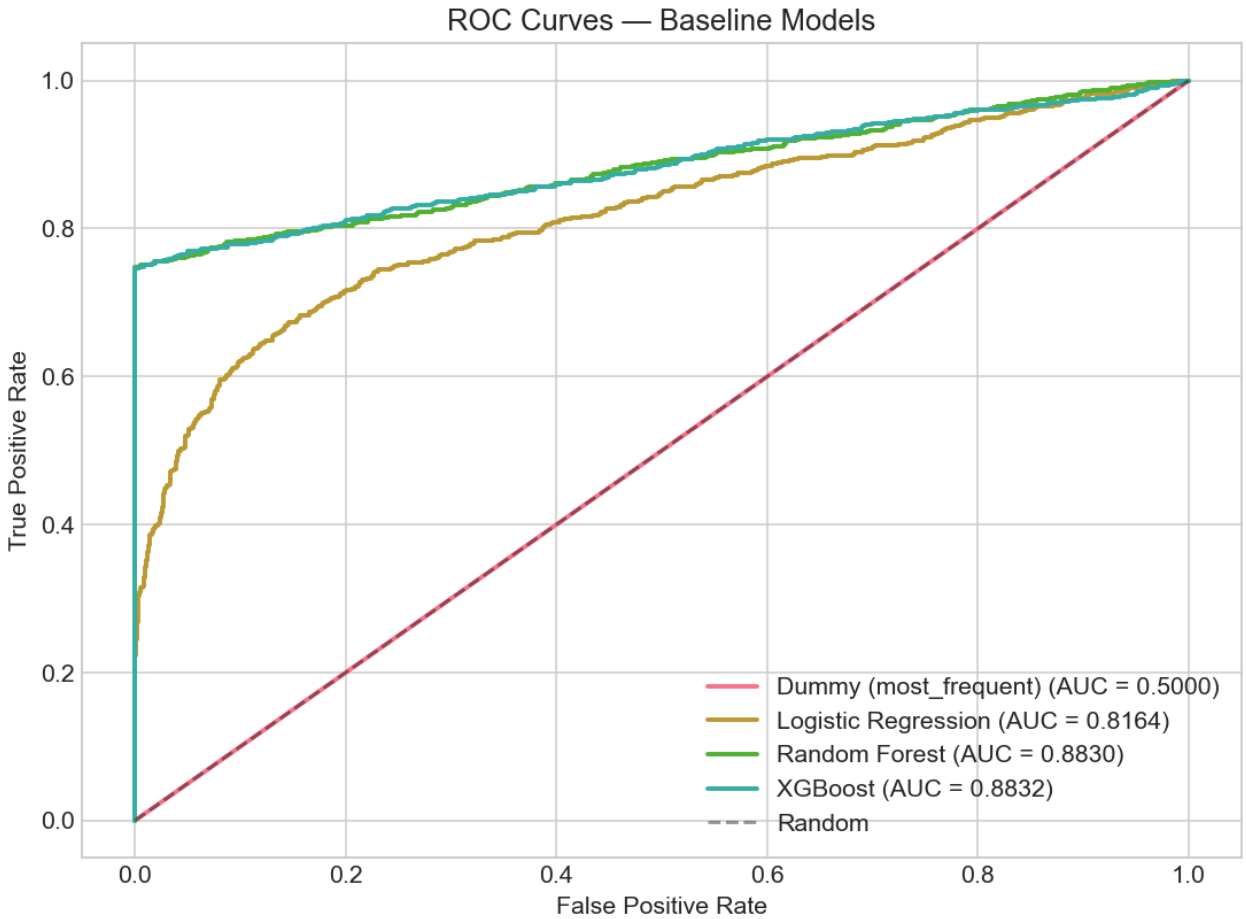

Figure 9: ROC curves of the four baselines on the test set. Random Forest and XGBoost lie almost on top of each other at AUC ~ 0.88.

The measured results on the held-out test set (1908 samples):

Model	Accuracy	F1	ROC-AUC	Train	Inference
Dummy (most_frequent)	0.5529	0.0000	0.5000	0.00 s	4.1 ms
Logistic Regression	0.7573	0.7297	0.8164	0.02 s	0.2 ms
Random Forest	0.8868	**0.8550**	0.8830	0.41 s	67.8 ms
XGBoost	0.8826	0.8507	**0.8832**	0.25 s	1.8 ms

Three findings came out of this step:

- The Logistic Regression at F1 = 0.73 is actually a useful signal. Three linear features (`failed_logins`, `login_attempts`, `ip_reputation_score`) already carry most of what is there. Our non-linear models had to do the other ~12 points of F1.

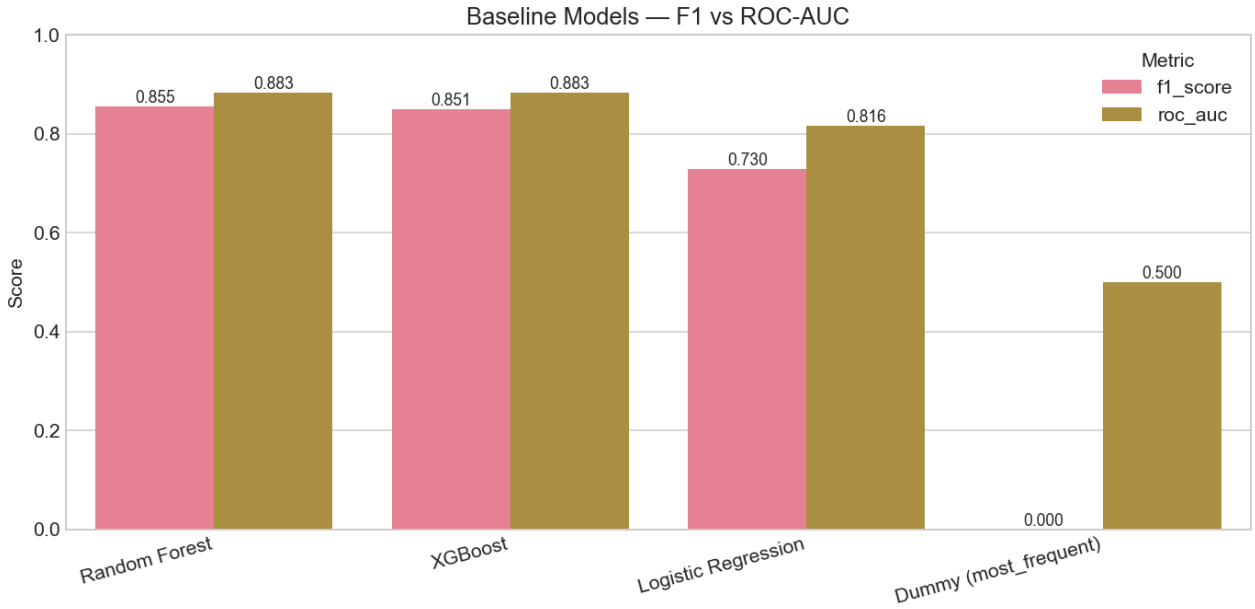

Figure 10: Barplot comparison of F1 vs AUC for the four baselines. Random Forest wins F1 (0.855); XGBoost wins AUC by a hair (0.8832 vs 0.8830).

- Random Forest and XGBoost are effectively tied. RF edges F1 by 0.004 and XGBoost edges AUC by 0.0002. We picked RF as the baseline champion on the F1 tie-break, but kept evaluating both through the rest of the project.
- **Precision on Attack is 1.00 for Random Forest**: zero false positives on the test set. Recall sits at 0.75, meaning we miss about a quarter of the attacks. This recall ceiling will recur across every model, and we discuss it as a property of the feature set, not of the algorithm.

3.2 Deep Learning (Figures 11 and 12)

We trained four DNN variants with Keras, all from the same parametric builder in `src/models/deep_learning.py`:

- **Baseline:** `Dense(128) → Dropout(0.3) → Dense(64) → Dropout(0.3) → Dense(32) → Dense(1, sigmoid)`, Adam (lr=1e-3), binary cross-entropy, `batch_size=32`, `validation_split=0.15`.
- **v1:** same architecture, `dropout = 0.5` (stronger regularisation).
- **v2:** same architecture, `batch_size = 128` (faster convergence).
- **v3:** deeper architecture `256 → 128 → 64 → 32`, `dropout = 0.3`, `batch_size = 32`.

Every variant used `EarlyStopping(patience=10, restore_best_weights=True)` and `ReduceLROnPlateau(fact` `patience=5)`. All four were logged to MLflow alongside the baselines so we could compare them side by side.

Results on the test set:

Variant	Accuracy	F1	AUC	Epochs	Train
DNN baseline (128-64-32, do=0.3)	0.8800	0.8470	0.8791	31	12.3 s
DNN v1 (dropout=0.5)	0.8810	0.8475	0.8813	30	11.2 s
DNN v2 (batch=128)	0.8816	**0.8493**	0.8816	34	**7.0 s**
DNN v3 (deep 256-128-64-32)	0.8805	0.8480	0.8819	26	12.1 s

The best DNN (**v2**, F1 = 0.8493) sits below Random Forest (F1 = 0.8550) and below XGBoost (F1 = 0.8507). The spread between the best and worst DNN variant is 0.23 F1 points, which is essentially noise. We read this as evidence that we hit the ceiling of the feature set, and that no

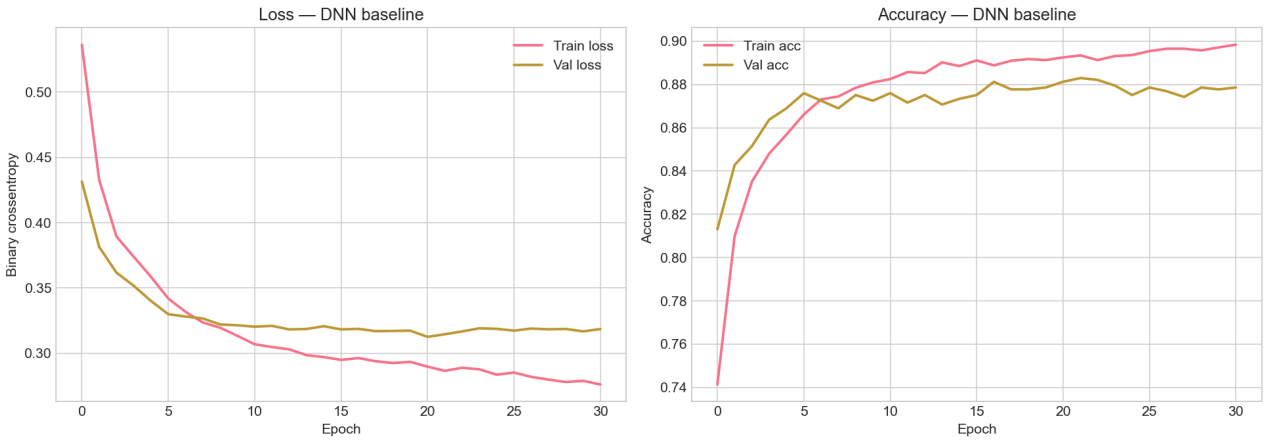

Figure 11: DNN baseline learning curves (train vs validation). EarlyStopping restored the best weights around epoch 22, before val_loss started creeping up.

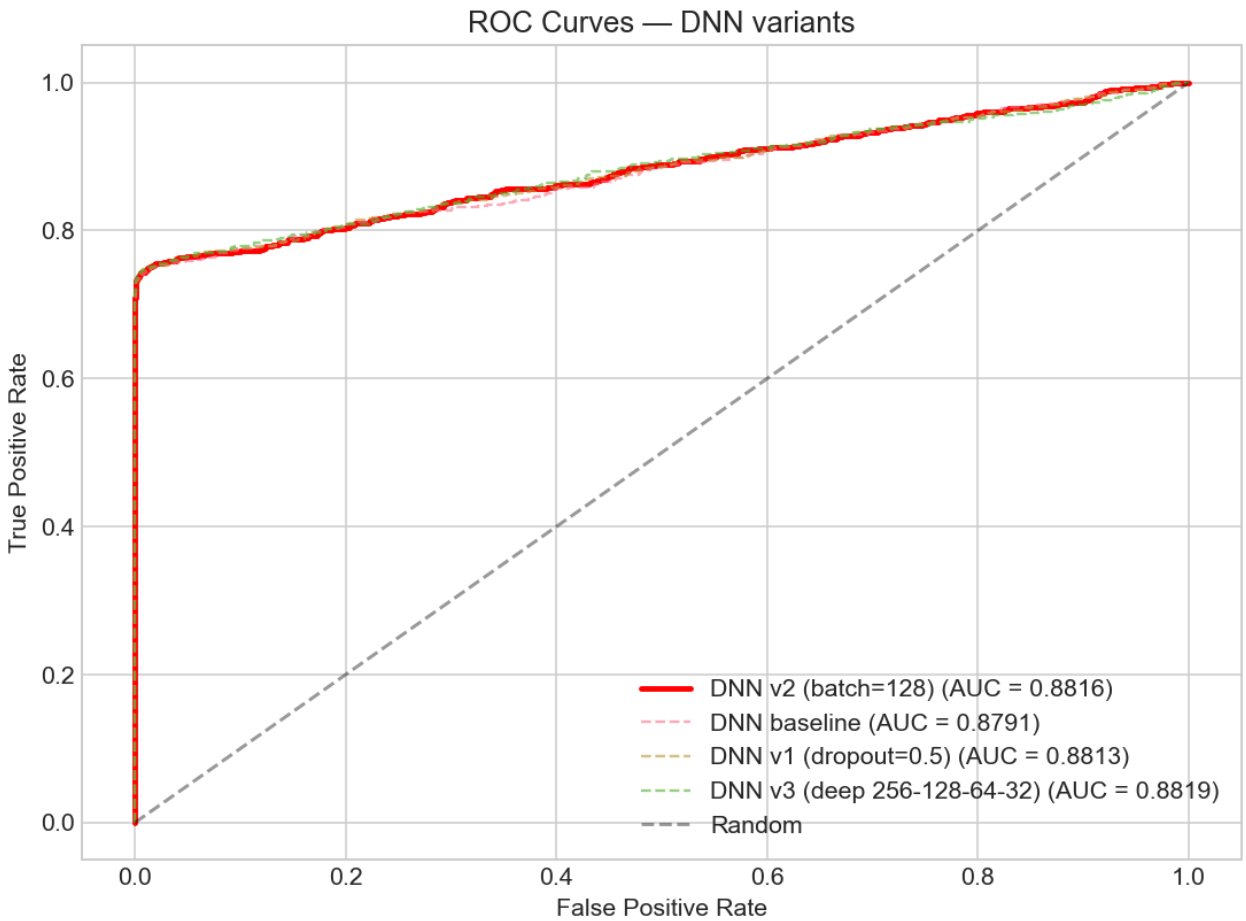

Figure 12: ROC curves for the four DNN variants. All four sit within a 0.003 AUC band, well inside the region Random Forest already occupies.

DNN architecture we could reasonably try would push F1 past ~0.85 on this data. The one DNN-specific takeaway was that `batch_size = 128` trained 1.7× faster than `batch_size = 32` for no loss in accuracy. We adopted it as the best DNN variant for everything that followed.

3.3 Ensembles

We tried a soft `VotingClassifier` (LR + RF + XGBoost) and a `StackingClassifier` with a Logistic Regression meta-learner, both trained on the same split with 5-fold CV for the meta-learner.

Ensemble	Accuracy	F1	AUC
Voting (soft, 3 models)	0.8863	0.8543	0.8762
Stacking (LR meta)	0.8857	0.8543	0.8840

Both ensembles gained 0.0007 F1 points at most over the single Random Forest, while tripling the inference time and doubling the artefact size. We concluded that ensembles did not pay their complexity cost on this problem, and we did not deploy them.

3.4 Champion selection

We deployed **Random Forest** as the champion. The justification:

- Highest F1 on the test set (0.855), the primary metric we picked at the start.
- Perfect precision on Attack (1.00 vs 0.99 for XGBoost): zero false positives, which matters operationally.
- Fast and cheap inference (0.019 ms per sample, around 50 000 predictions per second).
- TreeExplainer is roughly two orders of magnitude faster than KernelExplainer would be on a DNN. The live SHAP explanation in the UI (Section 6) would not run in real time on a DNN champion.
- No TensorFlow runtime required in production, so the deployed Docker image is smaller and simpler.

The champion model, the preprocessor, the default decision threshold (0.5), and the training metadata are bundled into `saved_models/best_model.joblib` by `scripts/export_models.py`. The FastAPI app loads the bundle once at startup.

---

4 Evaluation Metrics

4.1 Metrics chosen

Our primary metric is **F1 score on the Attack class**. Two reasons for this choice:

- The dataset is mildly imbalanced (1.24:1), so raw accuracy gives a slight reward to models that lean toward the majority class.
- In an IDS, a false negative (missed attack) is operationally worse than a false positive (extra alert). Recall on Attack is therefore the metric we watched most closely.

We also reported accuracy, precision, recall, ROC-AUC, PR-AUC, and inference time per sample in every MLflow run.

4.2 Full comparison table (Figure 20)

Across the 10 models we trained (4 baselines, 4 DNN variants, 2 ensembles):

Rank	Model	Accuracy	F1	Recall	Precision	AUC
1	**Random Forest**	0.8868	**0.8550**	0.7468	1.0000	0.8830
2	Stacking (LR meta)	0.8857	0.8543	0.7491	0.9938	0.8840
3	Voting (soft)	0.8863	0.8543	0.7456	1.0000	0.8762
4	XGBoost	0.8826	0.8507	0.7479	0.9861	0.8832
5	DNN v2 (batch=128)	0.8816	0.8493	0.7468	0.9845	0.8816
6	DNN v1 (dropout=0.5)	0.8810	0.8475	0.7397	0.9921	0.8813
7	DNN v3 (deep 256-128)	0.8805	0.8480	0.7456	0.9830	0.8819
8	DNN baseline	0.8800	0.8470	0.7433	0.9845	0.8791
9	Logistic Regression	0.7573	0.7297	0.7327	0.7267	0.8164
10	Dummy (most_frequent)	0.5529	0.0000	0.0000	0.0000	0.5000

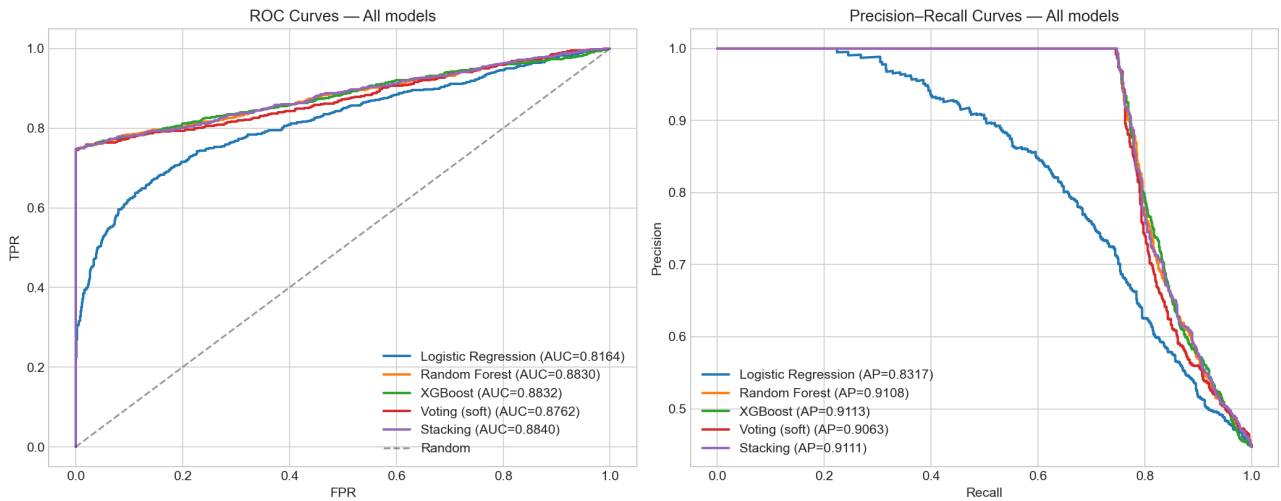

Figure 13: Final ROC and Precision-Recall curves with all models overlaid. The non-trivial models cluster in a band between F1 = 0.847 and 0.855.

Two structural observations come out of this table:

- Every non-trivial model lands within 0.01 F1 of every other one. We read this as an empirical case for a feature-set ceiling: with the 22 processed columns we have, no tuning or architectural change pushes F1 past ~0.855.
- Recall plateaus at 0.745 across every model. All eight of our top models miss approximately 25 % of the attacks. The interpretation we ended up with is that those attacks mimic normal sessions in the feature space we observe, and no classifier can separate them without additional inputs (packet payloads or host-level signals, for example).

4.3 Threshold analysis (Figure 21)

Because the default 0.5 decision threshold is arbitrary, we swept it on the XGBoost probabilities and computed F1 for each value.

The F1 curve is essentially flat. It peaks at t = 0.65 with a 0.0004 gain over the default. We kept 0.5 in production because the gain was not worth the extra parameter to justify, and because the "what threshold do you use?" question is already confusing for newcomers.

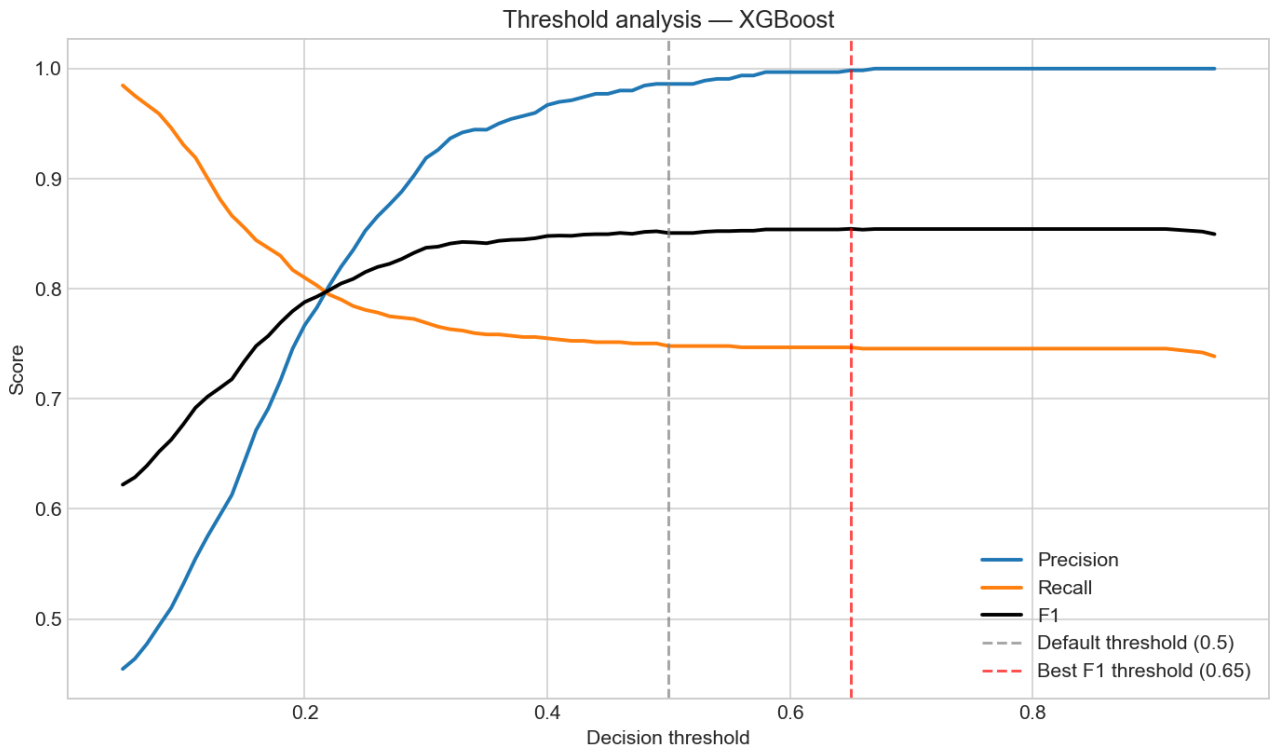

Figure 14: F1 versus decision threshold for XGBoost. The curve is flat between 0.4 and 0.75 and peaks at t = 0.65 with F1 = 0.851 — a marginal 0.0004 improvement over the default.

5 Interpretability and Explainability

5.1 Method choice

We used **SHAP TreeExplainer** on our tree-based champions (Random Forest and XGBoost). For the DNN we tried **KernelExplainer** on a 100-sample background, but the results were too noisy to be useful in practice, and Tree SHAP was orders of magnitude faster (about 1 ms vs about 10 s per prediction). Since our champion is Random Forest, Tree SHAP also happens to be what powers the live explanation in the deployed UI.

5.2 Global SHAP (Figures 13, 14, 19)

The two tree-based models ranked features almost identically:

1. `failed_logins` (mean|SHAP| = 1.66 on XGBoost; rank 1 on RF Gini)
2. `login_attempts` (1.03; rank 2)
3. `ip_reputation_score` (0.83; rank 3)
4. `browser_type_Unknown` (0.18; rank 6 on RF), confirming the EDA finding
5. `network_packet_size` (0.18; rank 9 on RF). SHAP surfaced it higher than Gini did, which suggests it participates mostly through interactions.

The convergence between SHAP and Gini told us the explanations were robust. We were not just looking at an artefact of one specific importance method.

5.3 Individual predictions (Figures 15, 16, 17)

We picked three representative sessions from the test set (a true positive, a false positive, and a false negative) and generated SHAP waterfall plots for each.

The false-negative case deserves special attention. Its features look entirely typical: Chrome browser, encrypted session, three login attempts, an IP reputation score of 0.45. Every numerical feature sits

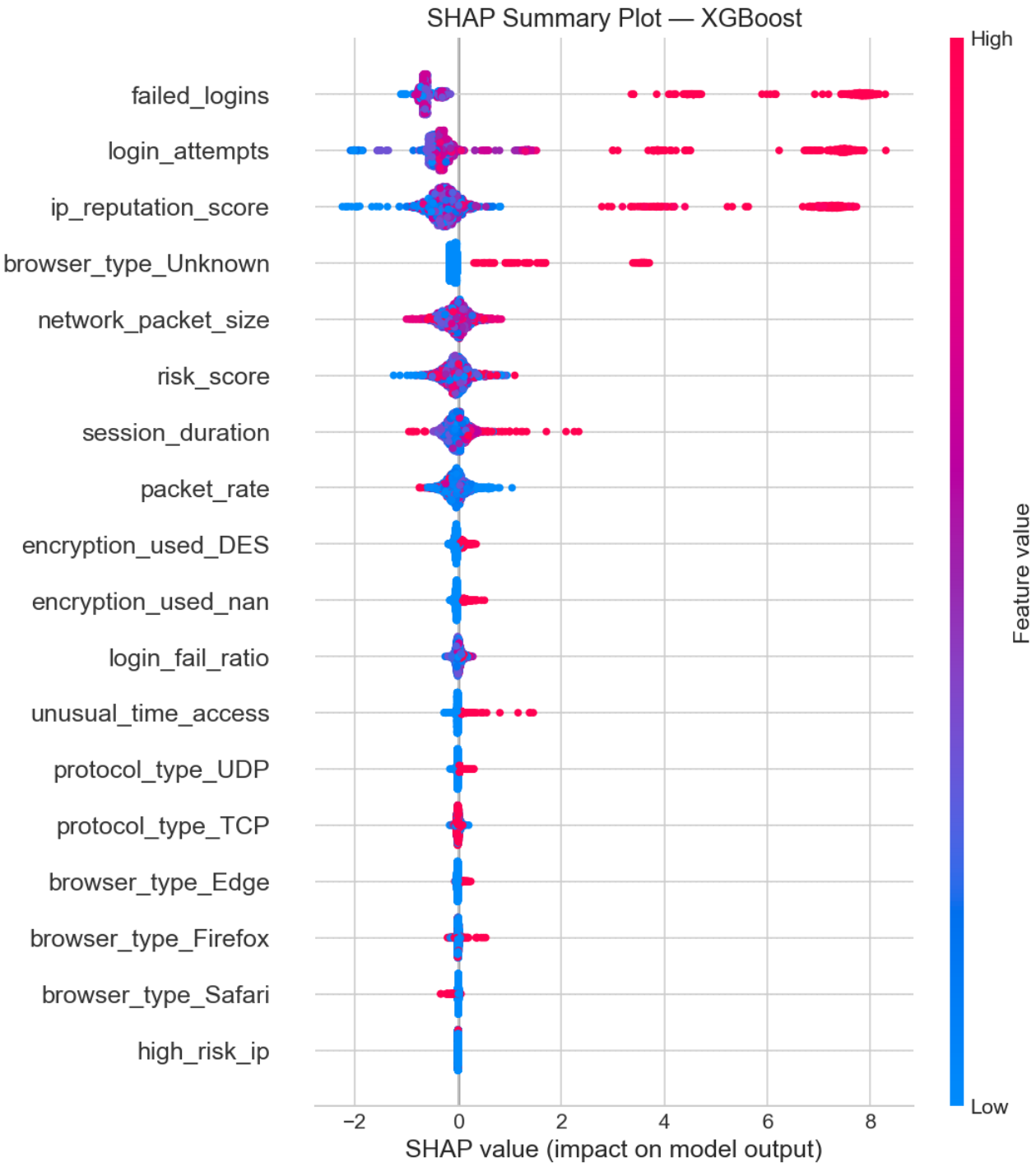

Figure 15: SHAP summary plot for XGBoost on the full test set. Points are samples; the y-axis ranks features by mean |SHAP|; the x-axis is each feature's contribution to pushing the prediction toward Attack (right) or Normal (left). Colour is the feature value (red = high, blue = low).

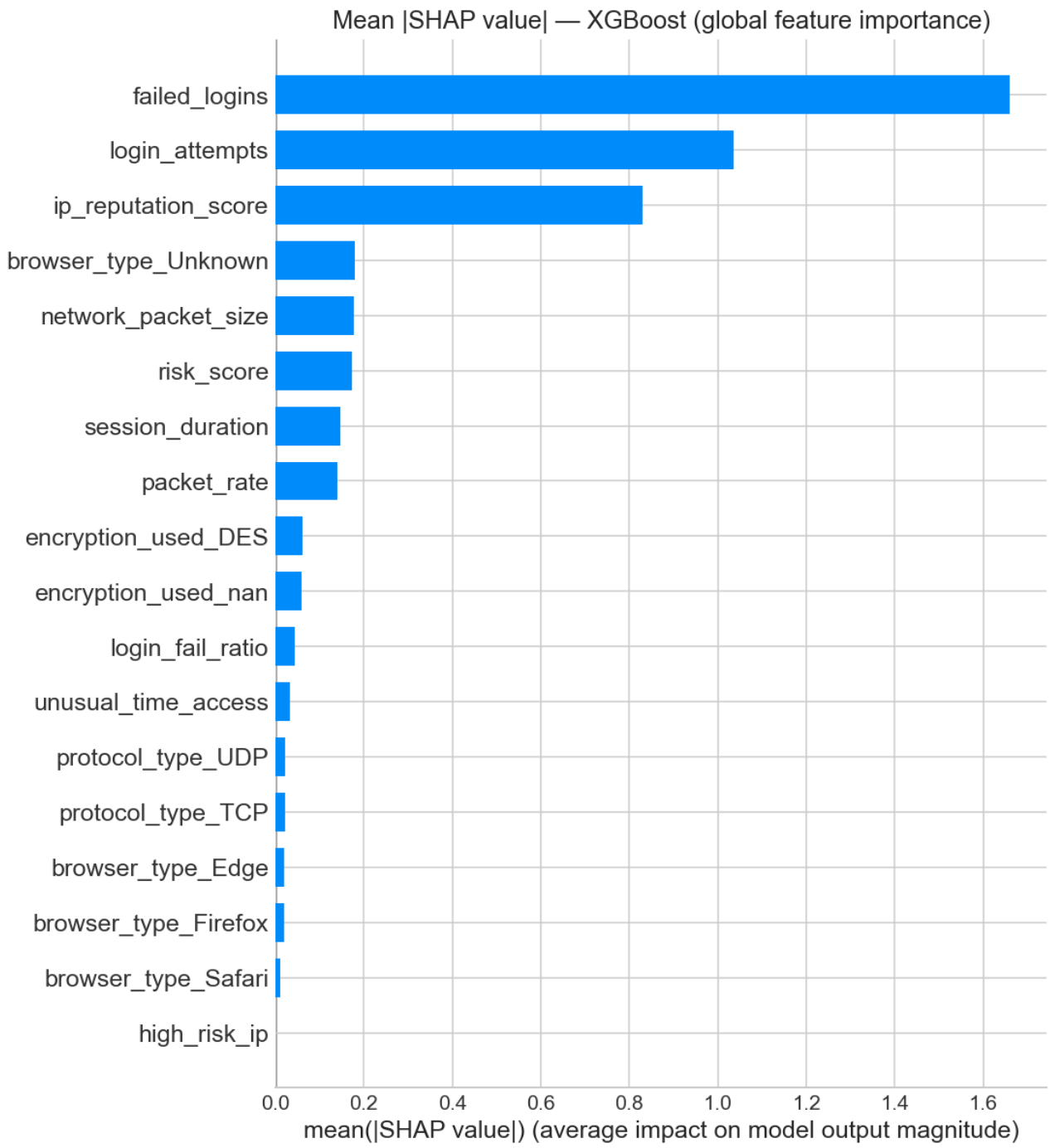

Figure 16: Mean absolute SHAP value per feature, XGBoost. failed_logins leads at 1.66, followed by login_attempts (1.03), ip_reputation_score (0.83), then a long tail.

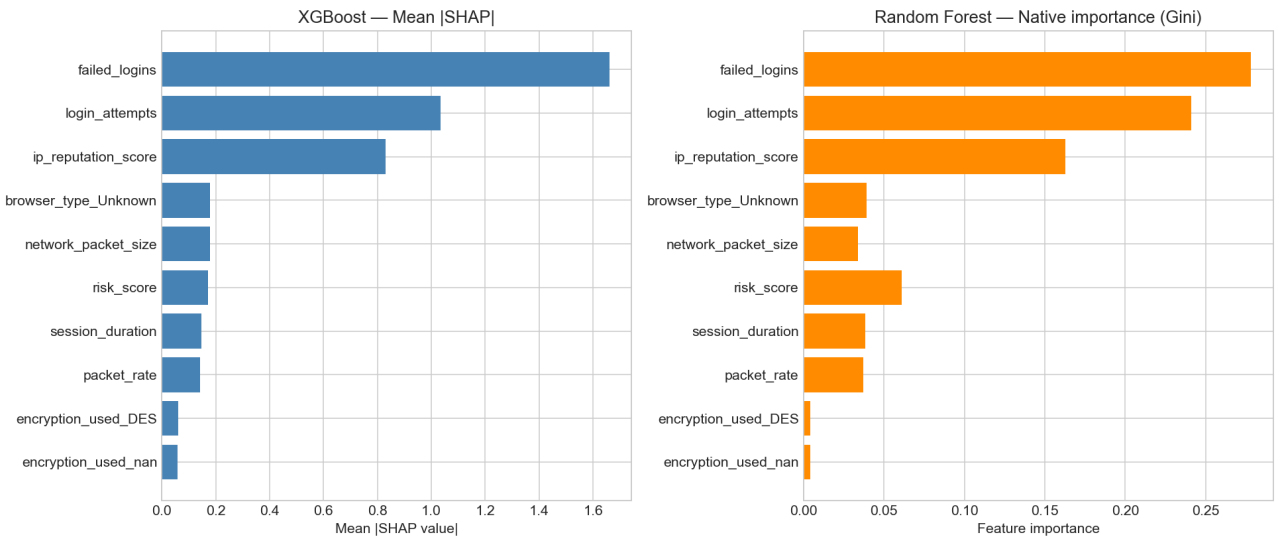

Figure 17: Side-by-side comparison of XGBoost SHAP importance and Random Forest Gini importance. The top three features are identical across methods, and browser_type_Unknown is consistently in the top six for both.

inside the densest part of the Normal distribution. We think attacks of this kind (an attacker using stolen credentials from a legitimate endpoint, for instance) are outside the reach of our data, and that no amount of model tuning would surface them. Detecting them would require payload inspection or host-level telemetry.

5.4 Feature dependence (Figure 18)

The plot confirms an intuitive interaction: failed logins from a clean IP are less suspicious than failed logins from a reputation-flagged IP. Both features contribute independently, and their combination contributes more than the sum of the parts. That is exactly the kind of interaction a non-linear model captures and a Logistic Regression cannot.

---

6 Deployment and Performance Tracking

6.1 Architecture

We deployed a **single Docker service**, `cybersecurity-ids-api`, running on port 8000. The service hosts:

- `GET /health`: Kubernetes-style readiness probe
- `GET /model/info`: model type, path, default threshold, training metadata
- `POST /predict`: JSON prediction endpoint
- `GET /` redirects (HTTP 307) to `/ui`
- `GET /ui`: the Gradio UI mounted on the same FastAPI app via `gr.mount_gradio_app(app, demo, path="/ui")`

We chose to use one service instead of one container for the API and one for the UI, for three reasons. The model stays loaded in memory once. The UI's prediction calls do not need cross-container HTTP hops. And the surface of things that can fail to start is halved. MLflow is available as an optional second service behind `--profile tools`, so `docker compose up -d` only launches the evaluated service.

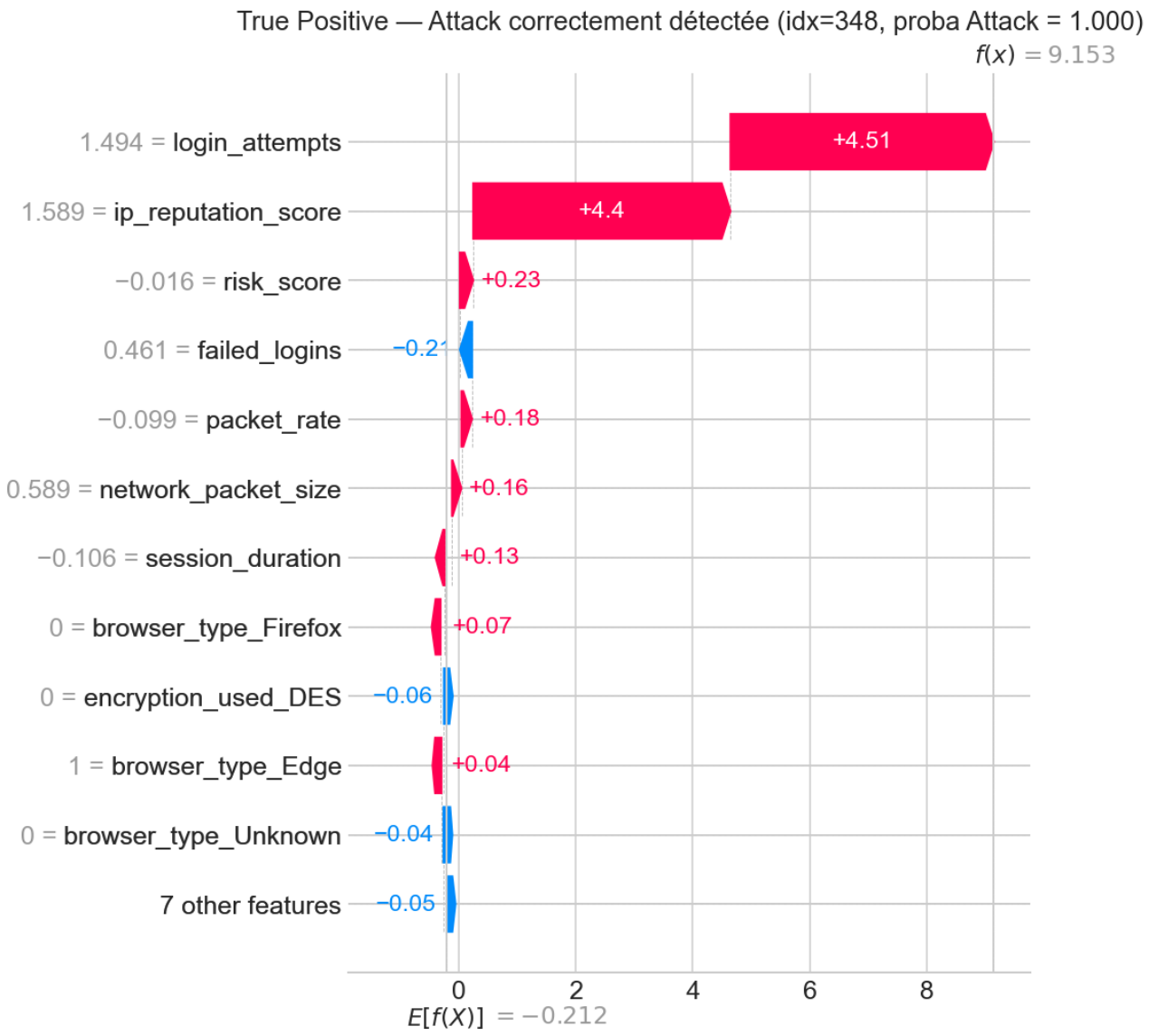

Figure 18: True Positive (idx=348, predicted probability 0.9999). The attack is obvious: high failed_logins, browser=Unknown, elevated ip_reputation_score all push toward Attack.

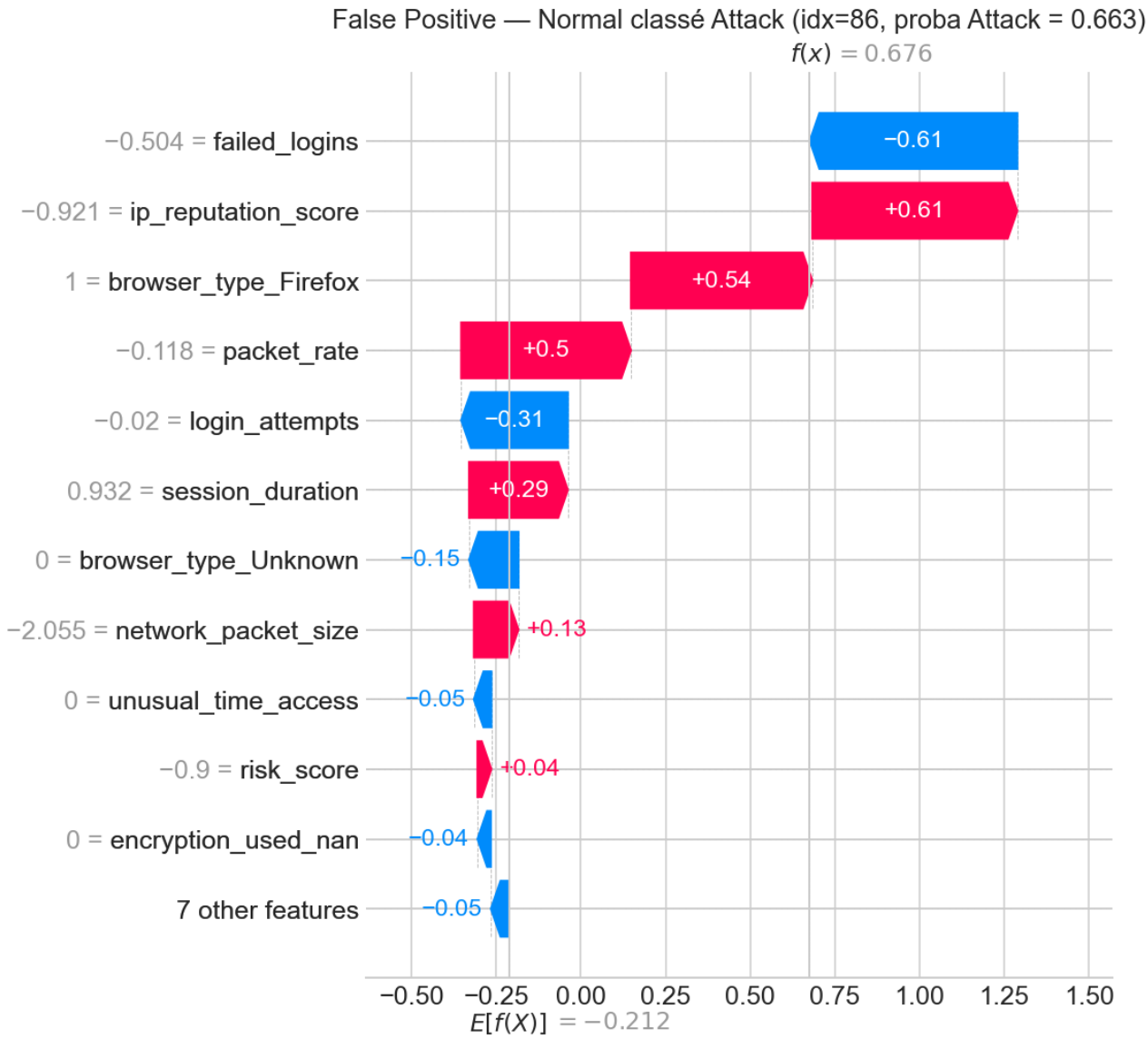

Figure 19: False Positive (idx=86, predicted probability 0.663). The combination of failed_logins and browser=Unknown makes the model over-weight; the session was actually Normal. A clear case where the model's prior about Unknown browsers is miscalibrated.

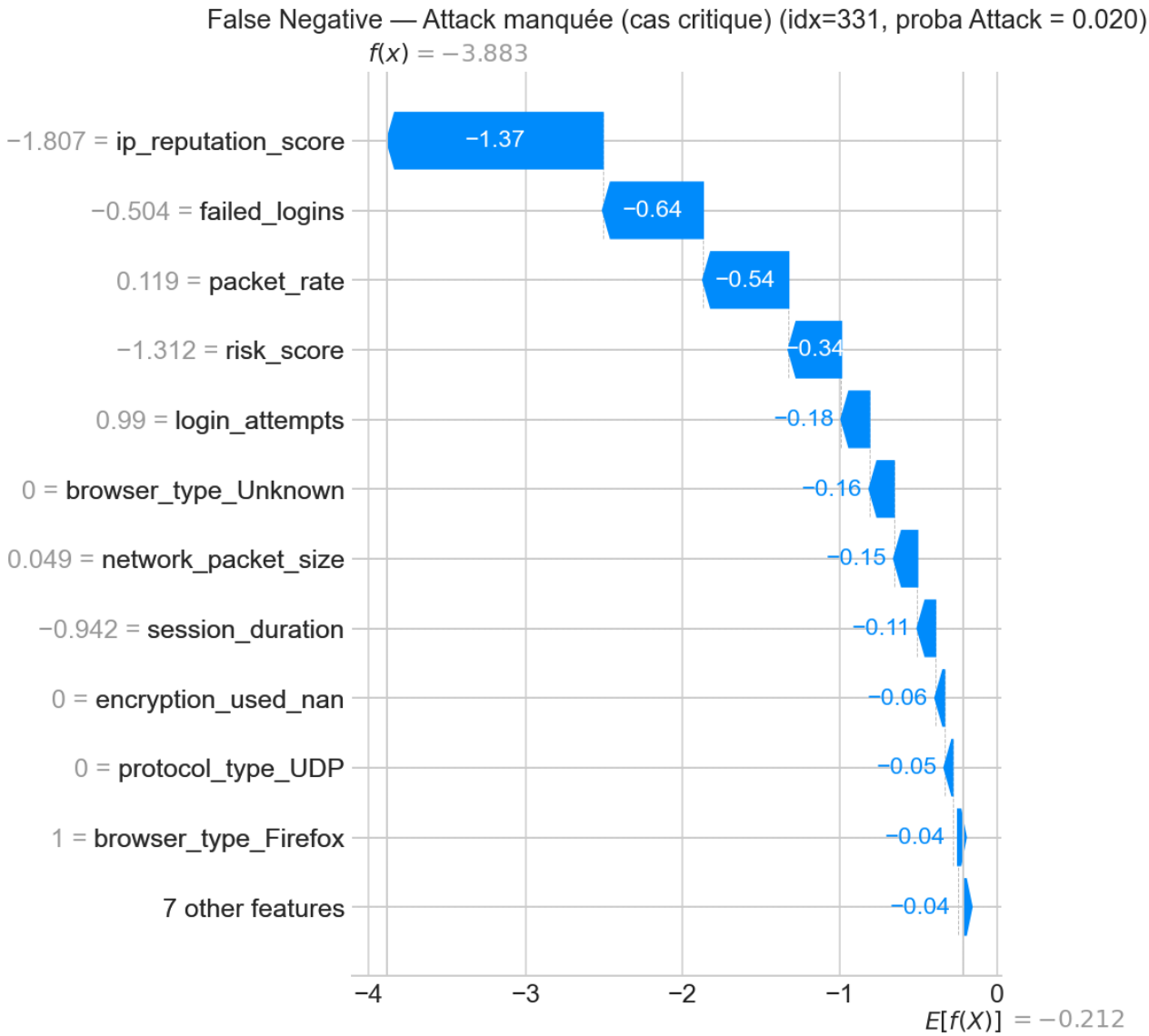

Figure 20: False Negative (idx=331, predicted probability 0.020). An attack that mimics a normal profile: known browser, encrypted session, modest login attempts, unremarkable IP reputation. The kind of attack our feature set cannot catch.

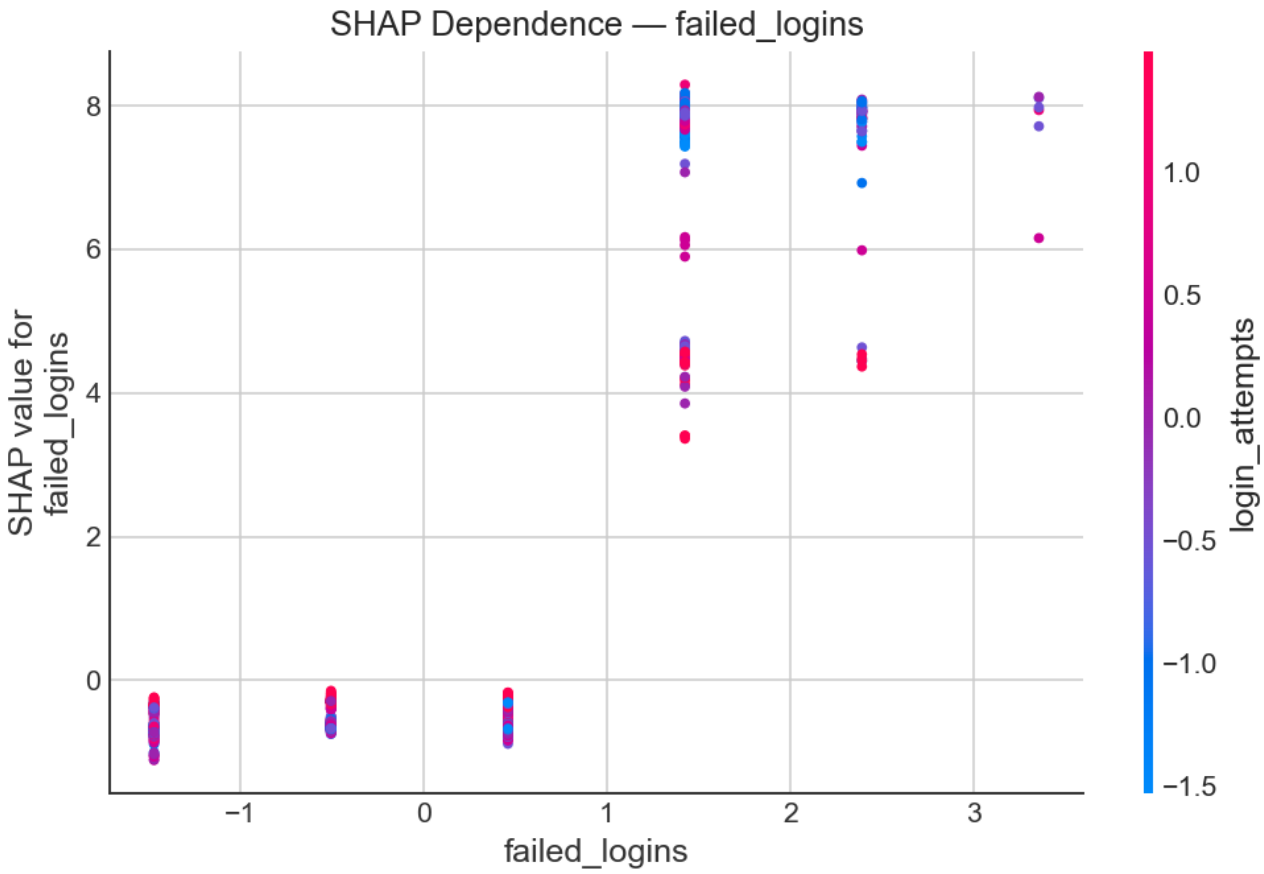

Figure 21: SHAP dependence plot for failed_logins (XGBoost), coloured by ip_reputation_score. The relationship is monotonic: more failed logins push toward Attack, and the effect is amplified when ip_reputation_score is high.

6.2 Champion bundle and startup

At container start, the FastAPI `startup` event loads `saved_models/best_model.joblib`. That bundle is a dict containing the champion model, the fitted preprocessor, the champion name, the default decision threshold, and training metadata. A second hook loads the four comparison models (LR, RF, XGBoost, DNN v2) from `saved_models/*.joblib` and `dnn.weights.h5`. If any of those is missing, the comparison section of the UI falls back gracefully to whatever was successfully loaded.

The DNN was saved as **weights only** (`dnn.weights.h5`) plus a minimal `dnn_arch.joblib` describing `n_features`. The API reconstructs the architecture at load time and calls `load_weights`. We adopted this pattern after running into a cross-version failure with the `.keras` format when we tried to load a host-saved model inside the container.

6.3 The Gradio UI: live SHAP and 4-model comparison

The UI has three parts on one page:

1. A 9-feature input form with sliders, dropdowns, and three one-click presets (Normal traffic, Obvious attack, Edge case).
2. The champion verdict in a coloured banner: label, probability, and risk level (LOW / MEDIUM / HIGH / CRITICAL).
3. A SHAP waterfall for the champion's prediction, generated on the fly (around 100 ms per call thanks to the cached TreeExplainer). It shows the top ten contributing features, with red bars pushing toward Attack and blue ones pushing toward Normal.
4. A "All 4 candidate models on this input" section displaying four cards side by side. Each card shows the model's probability on this sample, its prediction, its measured inference time, and (below a dashed separator) its **F1 on the full test set**. The best F1 is highlighted in green and bold. A banner below the cards reads CONSENSUS, PARTIAL, or DISAGREEMENT based on the spread across models, and reminds the reader that the champion is chosen on global F1, not on the current sample.

This last piece is the core of our demo story. On `Obvious attack`, all four models cluster around 0.99 to 1.00. On `Edge case`, Logistic Regression sits at exactly 0.499 (the linear boundary passes through the sample) while the three non-linear models all say Normal at 0.13 to 0.18. The live visual makes the argument of Section 3 tangible in three seconds.

6.4 `docker compose up -d` and the reproducibility contract

The full deployment contract is:

```
cd repo/docker
docker compose up -d
# wait ~7 seconds for the healthcheck to pass
# open http://localhost:8000 in a browser
```

No `pip install`, no model training, no manual file copying. `saved_models/` is committed (three classical `.joblib`, one `.h5`, one `.joblib` arch descriptor, one metadata file, and one champion bundle), so that `docker compose up -d` works from a fresh clone or from the ZIP we submitted, without any preparatory step.

Two Dockerfile decisions worth mentioning:

- **Ordered COPY.** `requirements.txt` is copied before `src/`, so the slow pip install layer (around 3 minutes for TensorFlow, XGBoost, SHAP, MLflow, Gradio) is cached when only our code changes. This takes the rebuild time from about 3 minutes to about 5 seconds for iterative development. We learned this trick the hard way after a few painful minutes-long rebuilds.
- **Read-only model mount.** The container mounts `saved_models/` as `:ro` because the API never writes model files, and we wanted to keep that path read-only.

6.5 MLflow tracking

Every training run in notebooks 02, 03 and 05 calls `mlflow.start_run()` inside the `cybersecurity-ids` experiment. We logged:

- **Parameters:** model name, all hyperparameters (stringified to avoid non-JSON-serialisable callables in XGBoost), preprocessing configuration, random seed
- **Metrics:** accuracy, precision, recall, F1, ROC-AUC, PR-AUC, train time, inference time
- **Per-epoch metrics for the DNN:** `loss`, `val_loss`, `accuracy`, `val_accuracy`
- **Artifacts:** confusion matrix PNG, serialised model

This gave us ten runs across the project that we can sort by F1 in the UI and compare side by side. We deliberately kept MLflow out of the `docker compose up -d` default path for two reasons. First, the evaluation only touches the one service that serves the model. Second, the run `meta.yaml` files contain absolute Windows paths for artefacts, which would not resolve inside a Linux container. MLflow is available locally via `mlflow ui` from the `repo/` directory to inspect the runs.

---

7 Reflections and Observations

7.1 Challenges faced

- **`sklearn` cross-version drift broke Logistic Regression loading.** The champion bundle was first serialised with `scikit-learn` 1.8.0 on a team member's laptop. The container had 1.7.2. Random Forest unpickled with a warning but ran fine. Logistic Regression raised `AttributeError: 'LogisticRegression' object has no attribute 'multi_class'` because the attribute had been removed in 1.8. We pinned `scikit-learn==1.7.2` in `requirements.txt` and retrained every model in that environment. The incident cost us about an hour and taught us that "warnings we ignored" in the notebook were hiding a real inference-time failure.

- **`.keras` model format failed to deserialise cross-version.** The DNN, saved with `tf.keras` on the host, refused to load inside the container with an error about an unrecognised `quantization_config` argument on `Dense` layers. The container and host both had TensorFlow 2.21.0, but the Keras internal serialisation format had evolved between minor versions. We switched to weights-only (`dnn.weights.h5`) plus a tiny architecture descriptor, and we now rebuild the model in-place at load time. This was the most portable way we found to ship a Keras model across environments.

- **MLflow `meta.yaml` files carry absolute paths.** When we tried to spin up the MLflow UI inside Docker to show the runs live, we noticed that every run recorded `artifact_uri: file:C:\Users\...\mlruns\...`, which does not exist in a Linux container. The UI would list the runs and metrics but fail on the artefact preview. We moved MLflow to an optional profile and recommended running `mlflow ui` directly on the host for inspection.

- **Gradio 6 deprecated `theme=` on the `Blocks` constructor.** A minor warning, harmless in practice but noisy at startup. We left it as-is. Moving theme to `launch()` would have forced us to un-mount from FastAPI.

7.2 Key takeaways

- **Stop tuning when you hit a plateau, look at the data instead.** Once every non-trivial model landed within 0.01 F1 of the others, we stopped tweaking hyperparameters and started looking at the recall ceiling itself. Reporting *why* the metric plateaus at 0.745 is a more useful contribution than a 0.001 F1 improvement from another GridSearchCV sweep.

- **Engineered features earn their place or they do not.** Three of our four engineered features ranked in the top six by correlation with the target. `packet_rate` did not. We kept it anyway because it is cheap to compute and a tree model can still exploit it through interactions.
- **Trees beat DL on tabular data of this size.** We trained 4 DNN variants with proper regularisation, and none of them matched Random Forest. This is consistent with the recent literature (Grinsztajn et al. 2022) and with common experience. DL shines on unstructured data (images, audio, text) where automatic feature extraction is the whole value proposition. On a 22-column tabular problem with engineered features that already do the heavy lifting, a gradient-boosted or bagged ensemble is the right tool.
- **Interpretability belongs in the deployed product.** A SHAP waterfall that shows up *inside the prediction UI* makes the explainability argument every time a user clicks Predict. A SHAP summary plot in a static report is a one-off the reader forgets.
- **The cost of a feature is also what it adds to the demo story.** Our 4-model comparison costs us about 300 ms per prediction and two extra model loads at startup. It is not the highest-performing addition we could have made. But it lets us defend our champion choice in the UI without any slide. The demo user sees *live* that every non-trivial model converges to about 0.85 F1 on the full test set, and that the champion is just the best one in a close race.

7.3 Limits and future work

- **Recall ceiling of 0.745 is fundamental, not algorithmic.** Lifting it would require new inputs (packet payload features, host telemetry, or temporal context such as session sequences). Anything we could have tried on the existing features would be inside the noise.
- **The Unknown-browser prior can misfire.** Figure 16 (false positive) shows a Normal session flagged Attack because of a combination of browser=Unknown and elevated failed_logins. A threshold adjustment or a second-stage classifier that explicitly re-evaluates Unknown cases could raise precision further, at a small cost to recall.
- **Continuous retraining.** The MLflow + Docker setup we built is already the infrastructure for a retraining pipeline. A cron job that re-runs `scripts/export_models.py` on fresh data would produce a new bundle, and the API would pick up the new bundle on restart with no code change. We did not wire the cron job, but everything else is in place.
- **Explainability for the DNN.** We used KernelExplainer and found it too slow to be useful in a UI. DeepExplainer would be a faster alternative to try, at the cost of a TF dependency in production. With Random Forest as the champion, this is a "nice to have", not a blocker.

---

8 Conclusion

We set out to build an interpretable, deployable intrusion detection model. The final deliverable is closer to "defensible" than "best-in-class", and we think that is the right mode for this problem and this data. The dataset's feature set imposes a performance ceiling around F1 = 0.855 and recall = 0.745, and within that ceiling all four of our non-trivial models converge to the same place. We deployed the one that was fastest and simplest to serve, backed the choice with four candidate models running side by side in the UI, and made every prediction self-explanatory with a live SHAP waterfall. The full project launches with `docker compose up -d` and opens at `http://localhost:8000`.

Beyond the F1 number, the part of the project we would want to highlight is the reasoning around why it plateaus where it does, and the UI that surfaces that reasoning on every click.

Project files submitted in this ZIP: `repo/notebooks/01-05` (EDA through final comparison), `repo/src/` (pipeline modules, FastAPI service, Gradio UI), `repo/scripts/export_models.py` (reproducible training), `repo/saved_models/` (pre-fitted artefacts), `repo/docker/` (Dockerfile +

docker-compose.yml), `repo/reports/figures/` (21 figures plus confusion matrices), `repo/README.md`, `repo/requirements.txt`.